SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA0313	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Athivaram Jayaprakash	**Reg. No.:**	192425311
Section / Batch:	AIML / 2024	**Date:**	

Code of Conduct

I **Athivaram Jayaprakash (192425311)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: A.Jayaprakash

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Part of Speech Tagging	Morphological decomposition of analyzing, analysis, and analytical; identification of roots, affixes, inflectional and derivational forms, and normalization for indexing.	Q1
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q2
Counting Words, English Word Classes, Part of Speech Tagging	Morphology-based normalization of govern, government, and governance for topic modeling, clustering, and lexical standardization.	Q3
Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Morphological analysis of activate, activation, and reactivation; identification of derivational layers, word-class changes, and semantic shifts.	Q4
Rule-Based Part of Speech Tagging, English Word Classes, Part of Speech Tagging	Inflectional normalization of create, creates, and creating; extraction of root forms and classification of tense/aspect transformations.	Q5

QUESTION 1 – Morphological analysis of analyzing, analysis and analytical

1. Problem Understanding

The search engine receives three terms that belong to the same broad concept family. It should recognize their relationship for retrieval, but it must not pretend that all three words have exactly the same grammatical role. Analyzing is a verb form, analysis is a noun, and analytical is an adjective.

2. Aim

The aim is to create a rule-based morphological processor that identifies the base or lexical stem, affix information, transformation type and a shared indexing family key for the three input words.

3. Objectives

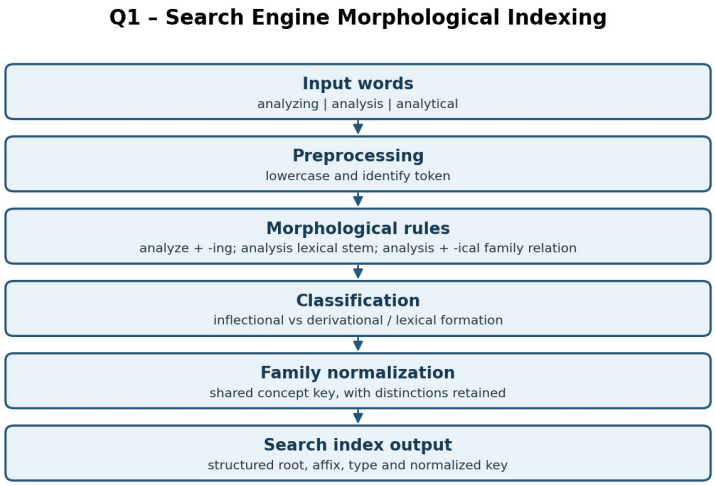
- Recognize the three input words and their word classes.
- Identify the inflectional ending in analyzing.
- Represent the lexical stem and formation of analysis and analytical accurately.
- Distinguish inflectional morphology from derivational or lexical word formation.
- Map the related terms to a shared retrieval family key while preserving their original forms.

4. Morphological Concepts

Analyzing is formed from analyze + -ing. The final e of analyze is dropped before -ing, so the surface form is analyzing. This is inflectional because it marks a participle/progressive verb form. Analysis is a noun with a Greek-derived stem pattern (analys- / analysis); it is not safely produced by simply stripping a regular English suffix from analyze.

Analytical is an adjective related to analysis/analytic and uses the derivational ending -ical. For indexing, the program uses the explicit family key ANALYSIS_FAMILY. This is a semantic grouping key, not a claim that all three have an identical surface root.

5. Architecture Diagram



5. Morphological Breakdown

Original word	Prefix	Root / base	Suffix	Transformation type	Meaning / grammatical change	Normalized output
analyzing	—	analyze	-ing (final e deleted)	Inflectional	Verb: present participle / progressive aspect	ANALYSIS_FAMILY
analysis	—	analys- / analysis (lexical stem)	-sis (historical lexical formation)	Lexical / derivational family member	Noun: process or result of examining	ANALYSIS_FAMILY
analytical	—	analysis / analyt-	-ical	Derivational	Adjective: relating to analysis	ANALYSIS_FAMILY

Linguistic caution: analysis and analytical are not reliably generated by a simple productive English suffix-stripping rule from analyze. The program uses an explicit lexical-family mapping for these specified words. The shared key is useful for search grouping, while POS and original token are retained.

6. Step-by-Step Algorithm

1. Read the three input words and convert them to lowercase.
2. Look up each word in the small rule/lexicon table.
3. For analyzing, identify analyze as the base and -ing as the inflectional suffix; record final-e deletion.
4. For analysis, record the lexical stem analys- and noun interpretation rather than applying a misleading English suffix rule.
5. For analytical, record the relation to analysis/analytic and the derivational suffix -ical.
6. Assign the shared family key ANALYSIS_FAMILY to support retrieval across the word family.
7. Print a structured row for each word with root, affixes, transformation type and normalized key.

7. Python Implementation

The code is a small, explicit rule-based morphological analyzer. A dictionary is appropriate here because analysis and analytical require lexical knowledge rather than a reliable general suffix rule.

```
words = ["analyzing", "analysis", "analytical"]

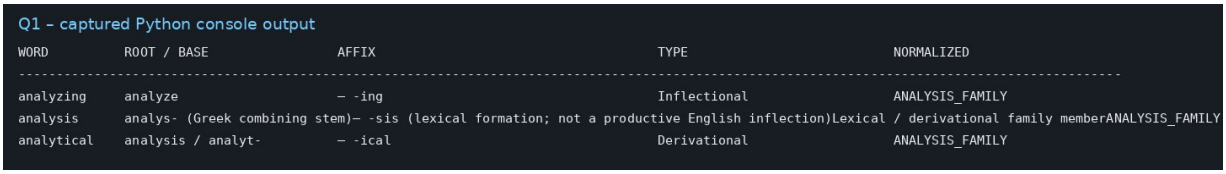
analysis = {
    "analyzing": {
        "root": "analyze",
        "prefix": "—",
        "suffix": "-ing",
        "type": "Inflectional",
        "meaning": "Verb: present participle / progressive form",
        "normalized": "ANALYSIS_FAMILY"
    },
    "analysis": {
        "root": "analys- (Greek combining stem)",
        "prefix": "—",
```

```
        "suffix": "-sis (lexical formation; not a productive English inflection)",
        "type": "Lexical / derivational family member",
        "meaning": "Noun: the process or result of examining something",
        "normalized": "ANALYSIS_FAMILY"
    },
    "analytical": {
        "root": "analysis / analyt-",
        "prefix": "—",
        "suffix": "-ical",
        "type": "Derivational",
        "meaning": "Adjective: relating to analysis",
        "normalized": "ANALYSIS_FAMILY"
    }
}

print(f"{'WORD':<14}{'ROOT / BASE':<28}{'AFFIX':<42}{'TYPE':<31}{'NORMALIZED'}")
print("-" * 145)
for word in words:
    item = analysis[word]
    affix = item["prefix"] + " " + item["suffix"]
    print(f"{word:<14}{item['root']:<28}{affix:<42}{item['type']:<31}{item['normalized']}")
```

8. Actual Sample Output

The following screenshot was generated from the output captured by executing the Python program in this environment.



9. Code Explanation

- `words` contains the exact three tokens specified in the assessment.
- `analysis` stores the root/lexical stem, affix description, transformation type, meaning and normalized family key for each token.
- The loop retrieves each record and prints it in columns.
- No regular expression is needed because the difficult analysis/analytical relationship is lexical and is better represented explicitly than guessed by suffix matching.
- The shared key improves recall for search but should be stored alongside the original word and part-of-speech information.

10. Result and Conclusion

The program identifies the grammatical and lexical structure of all three specified terms and assigns a shared indexing key. It avoids the incorrect assumption that analysis is simply analyze with a regular English suffix removed. This supports semantic grouping while retaining enough information to protect retrieval precision.

QUESTION 2 – Morphological parsing of disagree, agreement and agreeable

1. Problem Understanding

The sentiment platform receives related words in product reviews. The parser must identify their shared base agree and also preserve the semantic contribution of each affix. In particular, disagree expresses a negative or opposite relation, so removing dis- without retaining its feature could reverse the sentiment interpretation.

2. Aim

The aim is to parse prefixes and suffixes in disagree, agreement and agreeable, recover agree as the common base, classify each derivation and preserve its meaning for sentiment analysis.

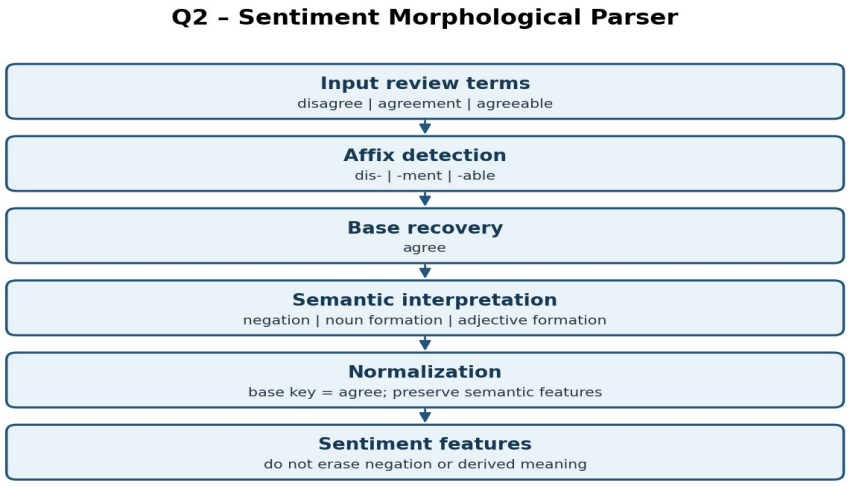
3. Objectives

- Detect the prefix dis- and suffixes -ment and -able.
- Recover agree as the shared base word.
- Classify the transformations as derivational.
- Explain the word-class and meaning changes.
- Preserve negation and derived semantic features after normalization.

4. Morphological Concepts

Disagree = dis- + agree. The prefix dis- adds opposition or negation and forms a verb with a different meaning. Agreement = agree + -ment; -ment forms a noun referring to a state, result or act. Agreeable = agree + -able; -able forms an adjective meaning pleasant or acceptable. All three transformations are derivational. Normalization to agree must not discard the semantic feature NEGATIVE for disagree.

5. Architecture Diagram



5. Morphological Breakdown

Original word	Prefix	Root/base	Suffix	Transformation type	Meaning / grammatical change	Normalized output
disagree	dis-	agree	—	Derivational prefix	Verb: to differ or not agree; negative/oppositional feature	agree + NEG
agreement	—	agree	-ment	Derivational suffix	Verb → noun: state/result/act of agreeing	agree
agreeable	—	agree	-able	Derivational suffix	Verb/base → adjective: pleasant or acceptable	agree

The normalized base is agree, but the feature NEG must be retained for disagree. Agreement and agreeable also remain semantically distinct even though they share the base.

6. Step-by-Step Algorithm

1. Read each word and convert it to lowercase.
2. Check whether the word begins with dis-. If yes, remove the prefix and record the negative/oppositional feature.
3. Otherwise, check for -ment and remove it to recover agree.
4. Otherwise, check for -able and remove it to recover agree.
5. Classify each detected affix as derivational.
6. Record the word-class and semantic change associated with the affix.
7. Output the common base and a semantic feature so that negation is not lost.

7. Python Implementation

This program uses explicit prefix/suffix rules and stores a separate semantic interpretation for each form.

```
words = ["disagree", "agreement", "agreeable"]

analysis = {
    "disagree": ("dis-", "agree", "—", "Derivational prefix",
                 "Negates or reverses agree; expresses non-agreement", "agree"),
    "agreement": ("—", "agree", "-ment", "Derivational suffix",
                  "Verb to noun: a state, result, or act of agreeing", "agree"),
    "agreeable": ("—", "agree", "-able", "Derivational suffix",
                  "Forms an adjective: pleasant or acceptable", "agree")
}

print(f"{'WORD':<14}{'PREFIX':<10}{'ROOT':<12}{'SUFFIX':<10}{'TYPE':<25}{'NORMALIZED'}")
print("-" * 105)
for word in words:
    prefix, root, suffix, kind, meaning, normalized = analysis[word]
    print(f"{word:<14}{prefix:<10}{root:<12}{suffix:<10}{kind:<25}{normalized}")
    print("  Meaning:", meaning)
```

8. Actual Sample Output

Captured from an execution of the program.

```
Q2 – captured Python console output
WORD          PREFIX    ROOT       SUFFIX    TYPE                    NORMALIZED
-------------------------------------------------------------------------------------------------------
disagree      dis-      agree      –         Derivational prefix     agree
  Meaning: Negates or reverses agree; expresses non-agreement
agreement     –         agree      -ment     Derivational suffix     agree
  Meaning: Verb to noun: a state, result, or act of agreeing
agreeable     –         agree      -able     Derivational suffix     agree
  Meaning: Forms an adjective: pleasant or acceptable
```

9. Code Explanation

- The dictionary maps each input word to prefix, root, suffix, transformation category, meaning and normalized base.
- The dis- rule is checked as a complete known form; the NEG interpretation is retained in the output.
- The -ment and -able suffixes are derivational because they form nouns and adjectives with new lexical meanings.
- The loop prints both a structured row and a separate meaning line.
- A general sentiment classifier should use context. Agreeable can be positive in some contexts, while agreement is not automatically positive or negative.

10. Result and Conclusion

The parser identifies the shared base agree and distinguishes the negative prefix from the two word-forming suffixes. It supports sentiment analysis by preserving semantic information instead of blindly replacing every word with agree.

QUESTION 3 – Morphology-based normalization of govern, government and governance

1. Problem Understanding

The news aggregation system receives articles that use govern, government and governance. These terms are closely related but do not have identical meanings: govern is an action, government is commonly an institution or governing body, and governance refers to the process, system or manner of governing. Normalization should help topic clustering without erasing these distinctions.

2. Aim

The aim is to build a morphology-based normalization module that detects derivational suffixes, identifies the common base govern, records derivational level and emits a shared lexical representation for topic modeling and clustering.

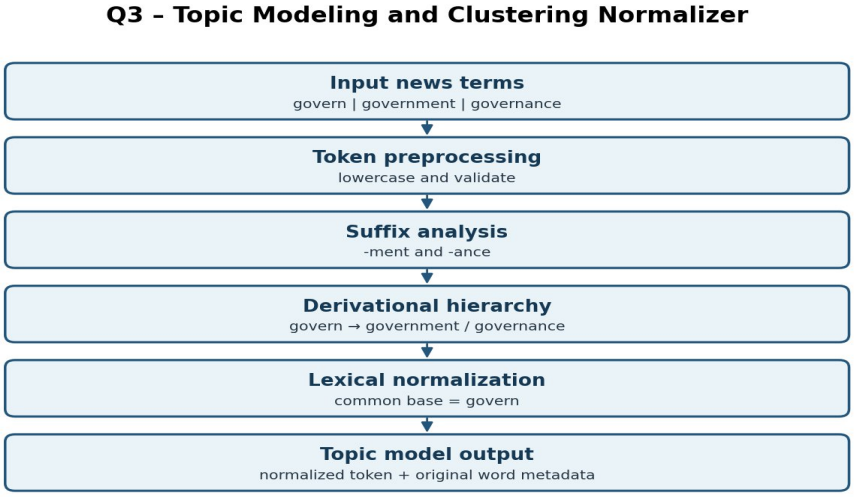
3. Objectives

- Identify govern as the common lexical base.
- Detect -ment in government and -ance in governance.
- Determine the derivational level of each term.
- Explain the word-class and semantic changes.
- Produce a common normalized representation while retaining original-word metadata.

4. Morphological Concepts

Govern is the base verb. Government is formed with -ment, which creates a noun and commonly refers to the governing body or system. Governance is formed with -ance, creating a noun that refers to the process, practice or system of governing. Both are derivational level 1 from govern in this simplified analysis. The normalized lexical base is govern; the original token and derived concept should remain available to the topic model.

5. Architecture Diagram



5. Morphological Breakdown

Original word	Prefix	Root/base	Suffix	Transformation type	Meaning / grammatical change	Normalized output
govern	—	govern	—	Base form	Verb: to rule, direct or control	govern
government	—	govern	-ment	Derivational, level 1	Verb → noun: governing body/system	govern
governance	—	govern	-ance	Derivational, level 1	Verb → noun: process/manner/system of governing	govern

Derivational hierarchy: govern → government; govern → governance. These are sibling derivations from the base, not a sequence in which government becomes governance.

6. Step-by-Step Algorithm

1. Read each input and lowercase it.
2. Check for the exact base form govern.
3. If the word ends in -ment, remove the suffix and recover govern; label it derivational level 1.
4. If the word ends in -ance, remove the suffix and recover govern; label it derivational level 1.
5. Assign the semantic description and noun/verb role.
6. Set the common lexical normalization to govern.
7. Print the root, affix, derivational level, meaning and normalized representation.

7. Python Implementation

The program uses a compact rule table because the scenario contains a known set of three related words. A larger news system should use a lexicon and POS-aware lemmatization.

```
words = ["govern", "government", "governance"]

analysis = {
    "govern": ("govern", "—", "Base form", "Verb: to rule or direct", "govern"),
    "government": ("govern", "-ment", "Derivational level 1",
                   "Verb to noun: governing body or system", "govern"),
    "governance": ("govern", "-ance", "Derivational level 1",
                   "Noun: process, manner, or system of governing", "govern")
}

print(f"{'WORD':<14}{'ROOT':<12}{'AFFIX':<10}{'DERIVATION':<23}{'NORMALIZED'}")
print("-" * 85)
for word in words:
    root, affix, level, meaning, normalized = analysis[word]
    print(f"{word:<14}{root:<12}{affix:<10}{level:<23}{normalized}")
    print("  Meaning:", meaning)
```

8. Actual Sample Output

Captured from an execution of the program.

Q3 – captured Python console output

```
WORD          ROOT        AFFIX     DERIVATION            NORMALIZED
--------------------------------------------------------------------------------
govern        govern      –         Base form             govern
  Meaning: Verb: to rule or direct
government    govern      -ment     Derivational level 1  govern
  Meaning: Verb to noun: governing body or system
governance    govern      -ance     Derivational level 1  govern
  Meaning: Noun: process, manner, or system of governing
```

9. Code Explanation

- The dictionary records each word's root, suffix, derivational level, meaning and normalized form.
- The -ment and -ance entries are derivational suffixes; they form nouns from the verb base.
- The loop displays the structural information and the meaning for each input.
- For topic modeling, use govern as the normalized token but retain the original word or a feature tag such as NOUN_INSTITUTION or NOUN_PROCESS when those distinctions matter.
- Simple suffix stripping can fail on unrelated words that happen to end in -ment or -ance; lexical validation is needed.

10. Result and Conclusion

The program identifies govern as the common base and recognizes two separate derivational nouns. It provides a shared token for clustering while documenting the different meanings of government and governance. This balances lexical standardization with semantic precision.

QUESTION 4 – Morphological parsing of activate, activation and reactivation

1. Problem Understanding

The document classification system receives activate, activation and reactivation. They share the base concept act, but each added derivational layer changes the meaning or grammatical role. The prefix re- in reactivation means that an activation happens again.

2. Aim

The aim is to identify the prefix, root and suffix layers in the three terms, describe their derivational sequence and word-class changes, and normalize them to a shared base concept for classification.

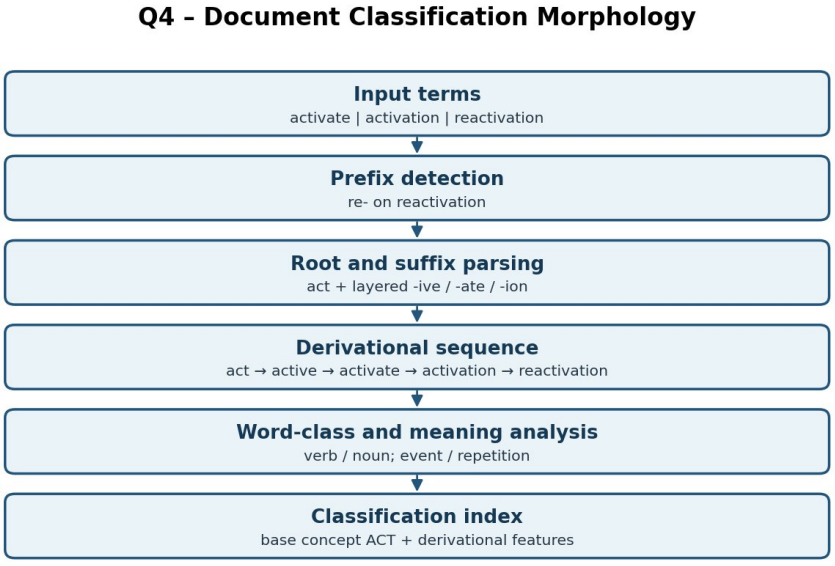
3. Objectives

- Recover act as the lexical root.
- Identify the layered derivational structure leading to activate and activation.
- Detect the prefix re- in reactivation.
- Explain the word-class and semantic changes at each layer.
- Produce a structured parse and common base representation.

4. Morphological Concepts

An accessible layered analysis is act → active → activate → activation. The -ive ending forms the adjective active; -ate forms the verb activate; -ion forms the noun activation. Reactivation adds re- to activation, meaning activation again. In this simplified classroom analysis, the shared root is act. The derivational sequence is more informative than treating the whole word as a single suffix removal.

5. Architecture Diagram



5. Morphological Breakdown

Original word	Prefix	Root/base	Suffix	Transformation type	Meaning / grammatical change	Normalized output
activate	—	act	-ive + -ate (layered)	Derivational	act → active (adjective) → activate (verb); cause to become active	ACT
activation	—	act	-ive + -ate + -ion	Derivational	activate (verb) → activation (noun): process/event	ACT
reactivation	re-	act	-ive + -ate + -ion	Derivational, layered	re- adds 'again'; noun: process of making active again	ACT + RE

The common concept key ACT supports grouping. The RE feature should be retained for reactivation because 'again' is a meaningful semantic contribution.

6. Step-by-Step Algorithm

1. Read the word and lowercase it.
2. Check whether it begins with re-. If so, record re- and remove it for the underlying form activation.
3. Match the remaining form against the known derivational sequence.
4. Recover act as the root and record the layers -ive, -ate and, where present, -ion.
5. Determine the final word class: activate is a verb; activation and reactivation are nouns.
6. Record the semantic effect of each layer, including the repeated-action meaning of re-.
7. Output the normalized base ACT while retaining RE as a feature for reactivation.

7. Python Implementation

This implementation uses a lexical rule table to represent the layered derivation clearly and avoid unsafe suffix stripping.

```
words = ["activate", "activation", "reactivation"]

analysis = {
    "activate": {
        "prefix": "—",
        "root": "act",
        "suffix": "-ive + -ate (layered formation: act → active → activate)",
        "sequence": "act (verb) → active (adjective) → activate (verb)",
        "class": "Verb",
        "meaning": "To cause something to become active",
        "normalized": "ACT"
```

```
    },
    "activation": {
      "prefix": "—",
      "root": "act",
      "suffix": "-ive + -ate + -ion",
      "sequence": "act → active → activate → activation",
      "class": "Noun",
      "meaning": "The process or event of making something active",
      "normalized": "ACT"
    },
    "reactivation": {
      "prefix": "re-",
      "root": "act",
      "suffix": "-ive + -ate + -ion",
      "sequence": "act → active → activate → activation → reactivation",
      "class": "Noun",
      "meaning": "The process of making active again",
      "normalized": "ACT"
    }
}

print(f"{'WORD':<17}{'PREFIX':<8}{'ROOT':<8}{'SUFFIX / LAYERS':<34}{'WORD CLASS':<12}{'NORMALIZED'}")
print("-" * 115)
for word in words:
    item = analysis[word]
    print(f"{word:<17}{item['prefix']:<8}{item['root']:<8}{item['suffix']:<34}{item['class']:<12}{item['normalized']}")
    print("  Sequence:", item["sequence"])
    print("  Meaning:", item["meaning"])
```

8. Actual Sample Output

Captured from an execution of the program.

Q4 – captured Python console output

```
WORD             PREFIX  ROOT    SUFFIX / LAYERS                  WORD CLASS  NORMALIZED
-------------------------------------------------------------------------------------------------------------------
activate         —       act     -ive + -ate (layered formation: act → active → activate)Verb        ACT
  Sequence: act (verb) → active (adjective) → activate (verb)
  Meaning: To cause something to become active
activation       —       act     -ive + -ate + -ion               Noun        ACT
  Sequence: act → active → activate → activation
  Meaning: The process or event of making something active
reactivation     re-     act     -ive + -ate + -ion               Noun        ACT
  Sequence: act → active → activate → activation → reactivation
  Meaning: The process of making active again
```

Result and Conclusion

The parser identifies act as the shared root, describes the derivational layers and preserves the repeated-action meaning of re-. This supports document classification by concept while retaining the semantic features that distinguish reactivation from activation.

QUESTION 5 – Inflectional normalization of create, creates and creating

1. Problem Understanding

The search optimization module receives the base verb create and two grammatical variants. It should map the forms to the same base for retrieval while identifying the grammatical information carried by each suffix.

2. Aim

The aim is to implement inflectional normalization for create, creates and creating by detecting suffix patterns, recovering create, classifying the grammatical form and producing a common normalized representation.

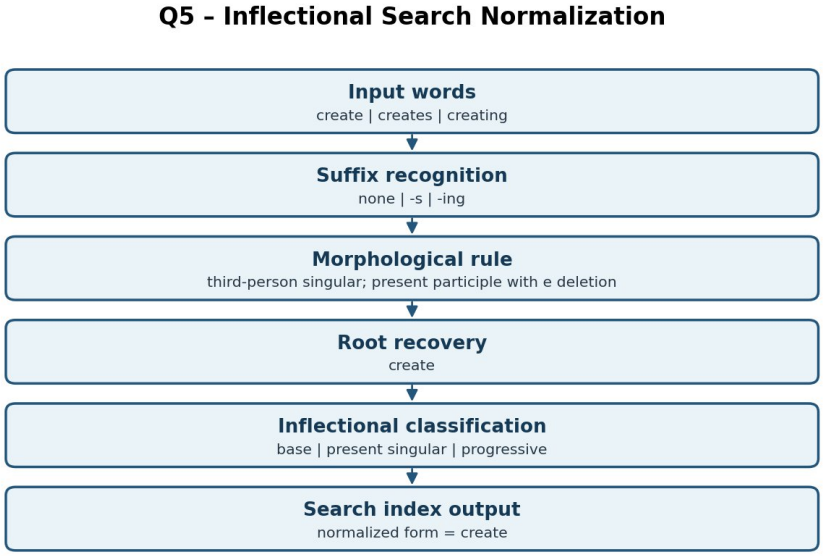
3. Objectives

- Recognize create as the base form.
- Detect -s in creates and classify it as third-person singular present.
- Detect -ing in creating and restore the dropped final e.
- Classify all changes as inflectional.
- Map all three words to create without changing their lexical meaning.

4. Morphological Concepts

Create is the base verb. Creates adds -s to mark third-person singular present agreement. Creating adds -ing to form a present participle or progressive construction; the final e in create is dropped before -ing. These are inflectional changes because they mark grammatical features and do not create a new lexeme or change the verb's word class.

5. Architecture Diagram



5. Morphological Breakdown

Original word	Prefix	Root/base	Suffix	Transformation type	Meaning / grammatical change	Normalized output
create	—	create	—	Base form	Verb lemma; no inflectional suffix	create
creates	—	create	-s	Inflectional	Third-person singular present tense / subject agreement	create
creating	—	create	-ing (final e deleted)	Inflectional	Present participle; used in progressive aspect	create

6. Step-by-Step Algorithm

1. Read each word and convert it to lowercase.
2. If the word is exactly create, classify it as the base form.
3. If the word ends in -s and matches creates, remove -s and recover create; label third-person singular present.
4. If the word is creating, remove -ing and restore the final e to recover create.
5. Classify the two suffix forms as inflectional.
6. Set the normalized output to create for all three words.
7. Display the original word, suffix, grammatical category, root and normalized form.

7. Python Implementation

The code explicitly handles the final-e spelling change in creating.

```
words = ["create", "creates", "creating"]

analysis = {
    "create": ("—", "Base form", "create", "create"),
    "creates": ("-s", "Third-person singular present", "create", "create"),
    "creating": ("-ing", "Present participle / progressive aspect; drop final e",
                 "create", "create")
}

print(f"{'WORD':<14}{'SUFFIX':<10}{'GRAMMATICAL CATEGORY':<52}{'ROOT':<12}{'NORMALIZED'}")
print("-" * 112)
for word in words:
    suffix, category, root, normalized = analysis[word]
    print(f"{word:<14}{suffix:<10}{category:<52}{root:<12}{normalized}")
```

8. Actual Sample Output

Captured from an execution of the program.

Q5 – captured Python console output

```
WORD          SUFFIX    GRAMMATICAL CATEGORY                                ROOT        NORMALIZED
-------------------------------------------------------------------------------------------------------------
create        –         Base form                                           create      create
creates       -s        Third-person singular present                       create      create
creating      -ing      Present participle / progressive aspect; drop final ecreate      create
```

9. Code Explanation

- The rule table assigns each word its suffix, grammatical category, root and normalized output.
- The -s rule represents third-person singular present agreement.
- The -ing rule represents a present participle/progressive form. The underlying e is restored in the root field.
- All three words remain verbs, so the change is inflectional rather than derivational.
- For open-ended input, use a lemmatizer or a more complete spelling-rule module rather than assuming every word ending in -s or -ing follows these rules.

10. Result and Conclusion

The program maps create, creates and creating to the common base create and identifies the grammatical feature expressed by each form. It supports search optimization by matching documents and queries that use different inflections of the same verb.